All these tests are evaluated in the same way, looking at the gain ratio (or, optionally, the gain) arising from the division of the training cases that they produce. It has also proved useful to introduce a further constraint: For any division, at least two of the subsets T_i must contain a reasonable number of cases. This restriction, which avoids near-trivial splits of the training cases, comes into play only when the set T is small. (The default minimum number of cases is 2, but this can be changed via an option as described later.)

2.4 Tests on continuous attributes

It might seem that tests on continuous attributes would be difficult to formulate, since they contain arbitrary thresholds. This is not so: The method for finding such tests can be described very concisely. This algorithm for finding appropriate thresholds against which to compare the values of continuous attributes was drawn to my attention by Paterson and Niblett [1982], although the same strategy appears also in Breiman *et al.* [1984].

The training cases T are first sorted on the values of the attribute A being considered. There are only a finite number of these values, so let us denote them in order as $\{v_1, v_2, \ldots, v_m\}$. Any threshold value lying between v_i and v_{i+1} will have the same effect of dividing the cases into those whose value of the attribute A lies in $\{v_1, v_2, \ldots, v_i\}$ and those whose value is in $\{v_{i+1}, v_{i+2}, \ldots, v_m\}$. There are thus only $m - 1$ possible splits on A, all of which are examined. (It might seem expensive to examine all $m - 1$ such thresholds, but, when the cases have been sorted as above, this can be carried out in one pass, updating the class distributions to the left and right of the threshold on the fly.)

It is usual to choose the midpoint of each interval as the representative threshold, the ith such being

$$\frac{v_i + v_{i+1}}{2.}$$

C4.5 differs in choosing the largest value of A in the entire training set that does not exceed the midpoint above, rather than the midpoint itself, as the threshold; this ensures that all threshold values appearing in trees and/or rules actually occur in the data.

To illustrate this threshold-finding process, we return briefly to the labor negotiation data discussed in Chapter 1 and the attribute wage increase first year. There are 15 distinct values of this attribute in the

40 training cases, with 14 associated thresholds, but the largest value is excluded because it splits off only 1 training case. Figure 2–3 shows the values of gain ratio and gain resulting from divisions based on the 13 possible thresholds. Both gain and gain ratio happen to peak at the same threshold in this instance, but the peak for gain ratio is more clearly defined. As the figure shows, the best threshold for the wage increase first year gives a division into values up to 2.5 on the one hand, and higher values on the other; this division gives a greater gain ratio value than any other test and so becomes the root of the decision tree.

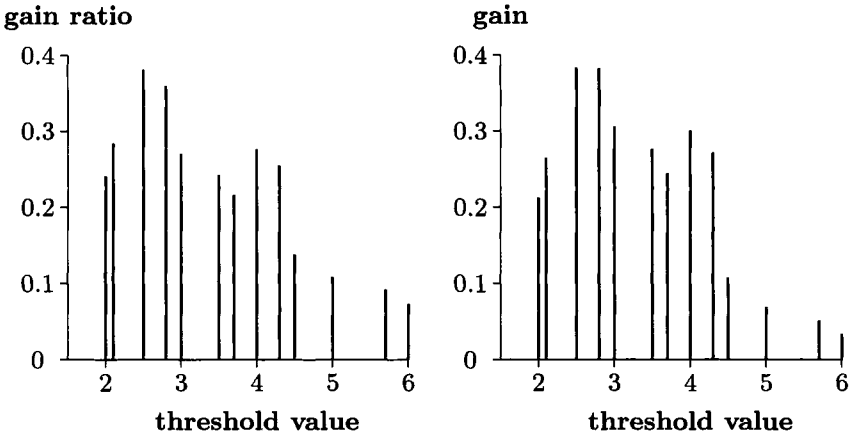


Figure 2–3. Gain ratio and gain as a function of threshold for the attribute wage increase first year, labor negotiation data.

One point should be made here: Whereas most of the operations required to construct a given decision tree grow linearly with the number of training cases, sorting d continuous values grows as $d \times \log(d)$. The time required to construct a decision tree from a large training set can be dominated by the sorting of continuous attributes as a prelude to examining possible thresholds on them. Several authors, notably Catlett [1991b], have discussed mechanisms for ameliorating this computational impost.